

Лабораторная работа 3. Transfer Learning и интерпретируемость сверточных нейронных сетей

В предыдущей лабораторной работе вы построили и обучили сверточную нейронную сеть с нуля, применили пакетную нормализацию, аугментацию данных и провели серию экспериментов по улучшению модели. В этой работе вы:

- Используйте **тот же датасет**, что и в предыдущей лабораторной (или выбираете новый, если хотите, — с соблюдением тех же ограничений). Рекомендуется выбрать другой датасет, если ассурасу превышает **95%**, чтобы transfer learning имел смысл.
- Сравните **лучший результат из предыдущей работы** (модель, обученная с нуля) с результатами transfer learning.
- Примените методы интерпретируемости, чтобы понять, **почему** модель принимает те или иные решения.

Если вы выбираете **новый датасет**, он должен удовлетворять тем же требованиям: многоклассовая классификация (≥ 5 классов), изображения не более 128×128 , объём от 5 000 до 100 000 изображений.

Подготовка данных

Используйте pipeline подготовки данных из предыдущей работы (включая аугментацию для обучающей выборки и нормализацию для всех выборок).

Важное дополнение: предобученные модели из `torchvision` обучены на ImageNet со следующими статистиками нормализации

```
imagenet_mean = [0.485, 0.456, 0.406]
imagenet_std = [0.229, 0.224, 0.225]
```

При использовании предобученных моделей **необходимо нормализовать данные именно с этими значениями**, а не со статистиками вашего датасета. Это связано с тем, что модели из `torchvision` обучались на датасете **ImageNet**, и их веса оптимизированы для входных данных с такими статистиками.

Также большинство предобученных моделей ожидают **3-канальные (RGB) изображения**. Если ваш датасет содержит grayscale-изображения, необходимо преобразовать их в трёхканальный формат. Проще всего это сделать, переведя изображение в формат **RGB**, при котором один и тот же канал будет продублирован три раза.

Пример pipeline подготовки данных:

```
transform = transforms.Compose([
    transforms.Resize((224, 224)),
    transforms.Lambda(lambda img: img.convert("RGB")),
    transforms.ToTensor(),
    transforms.Normalize(imagenet_mean, imagenet_std)
])
```

Преобразование `img.convert("RGB")` безопасно применять и для цветных изображений: если изображение уже является RGB, оно останется без изменений.

Большинство предобученных моделей обучались на изображениях размером **224×224**, поэтому рекомендуется использовать `transforms.Resize((224, 224))`.

Если исходный датасет содержит маленькие изображения (например, **32×32** или **64×64**, как в CIFAR-подобных датасетах), их обычно **масштабируют до 224×224** перед подачей в предобученную модель. Это стандартная практика при использовании transfer learning.

Сверточные сети могут работать и с меньшими размерами изображений (например, 64×64 или 128×128), однако при слишком маленьком входе модель может терять часть пространственной информации, что иногда ухудшает качество.

Задание 1. Обязательное для всех (максимальная оценка — 8)

Освойте два режима переноса обучения и сравните их с обучением модели с нуля. Первый режим — использование предобученной сети как фиксированного экстрактора признаков. Второй — полный fine-tuning.

Необходимо обучить и сравнить **три модели** на одном и том же датасете с одинаковой аугментацией и нормализацией (с поправкой на статистики ImageNet для предобученных моделей).

Модель А: Обучение с нуля (результат предыдущей работы)

Возьмите **лучшую модель из предыдущей лабораторной работы** (или обучите заново, если меняете датасет). Она служит базовой точкой для сравнения.

Если вы меняете датасет или размер изображений, переобучите модель из предыдущей работы на новых данных, чтобы сравнение было корректным.

Модель В: Предобученная модель с замороженными слоями

1. **Загрузите предобученную модель** из `torchvision.models`. Выберите одну из архитектур (например, `resnet18`, `resnet34`, `mobilenet_v3_large`, `efficientnet_b0`, `convnext_tiny`, `regnet_y_400mf` и другие на ваше усмотрение):

```
import torchvision.models as models

# или другая модель
model = models.resnet18(weights=models.ResNet18_Weights.DEFAULT)
```

2. **Заморозьте все параметры** model, чтобы они не обновлялись при обучении:

```
for param in model.parameters():
    param.requires_grad = False
```

3. **Замените классификационную голову** (последний полносвязный слой) на новую, соответствующую числу классов вашего датасета. Структура головы зависит от выбранной архитектуры:

- **ResNet:** заменяется `model.fc`
- **MobileNet / EfficientNet / ConvNeXt / RegNet:** заменяется `model.classifier[-1]`

Чтобы узнать структуру модели, выведите её: `print(model)`.

Новая голова может быть:

простым линейным слоем

```
nn.Linear(in_features, num_classes)
```

или небольшой нейронной сетью, например:

```
nn.Sequential(
    nn.Linear(in_features, 256),
    nn.ReLU(),
```

```
nn.Dropout(0.5),
nn.Linear(256, num_classes)
)
```

4. **Убедитесь**, что в оптимизатор передаются **только обучаемые параметры**:

```
opt = torch.optim.Adam(
    filter(lambda p: p.requires_grad, model.parameters()),
    lr=0.001
)
```

5. **Обучите модель** до сходимости, отслеживая train loss и val accuracy.

Модель C: Полный Fine-tuning

1. **Загрузите ту же предобученную модель** заново (с предобученными весами).
2. **Замените классификационную голову** аналогично Модели B.
3. **Не замораживайте параметры** - все слои должны быть обучаемыми. По умолчанию параметры в PyTorch имеют `requires_grad = True`, поэтому если вы загрузили модель заново, дополнительные действия не требуются.
4. **Используйте пониженный learning rate для backbone** и более высокий — для новой классификационной головы. Это можно реализовать через отдельные группы параметров в оптимизаторе:

```
opt = torch.optim.Adam([
    {'params': backbone_params, 'lr': 1e-4}, # малый lr для предобученных слоёв
    {'params': head_params, 'lr': 1e-3}, # нормальный lr для новой головы
])
```

Разделение параметров зависит от архитектуры. Для ResNet, например:

```
head_params = model.fc.parameters()
backbone_params = [p for name, p in model.named_parameters() if "fc" not in name]
```

5. **Обучите модель** до сходимости, отслеживая train loss и val accuracy.

Сравнение трёх моделей

Сведите результаты в таблицу:

Модель	Режим	Число обучаемых параметров	Время обучения (на эпоху)	Число эпох до сходимости	Val accuracy	Test accuracy
A	С нуля	...	...	...	...%	...%
B	Предобученная модель с замороженными слоями	...	...	...	...%	...%
C	Fine-tuning	...	...	...	...%	...%

Число обучаемых параметров можно посчитать так:

```
trainable_params = sum(p.numel() for p in model.parameters() if p.requires_grad)
total_params = sum(p.numel() for p in model.parameters())
print(f"Обучаемых:{trainable_params:,} / Всего:{total_params:,}")
```

Задание 2. Дополнительное (максимальная оценка — 10)

Исследуйте, какие признаки извлекает сверточная нейронная сеть, на что «обращает внимание» модель при классификации, и проанализируйте типичные ошибки.

Используйте **лучшую модель из Задания 1**. Все визуализации и анализ выполняются для неё.

Часть А. Визуализация фильтров первого сверточного слоя

1. **Найдите первый сверточный слой модели** и извлеките его веса.

Например, для **ResNet**:

```
first_conv = model.conv1
filters = first_conv.weight.detach().cpu()
print(filters.shape) # [out_channels, in_channels, H, W]
```

Если вы используете другую архитектуру, название слоя может отличаться. Чтобы узнать структуру модели, выведите её:

```
print(model)
```

2. **Визуализируйте фильтры** в виде сетки изображений.

Каждый фильтр — это маленькое изображение (обычно 3×3, 5×5 или 7×7).

Перед визуализацией нормализуйте значения в диапазон **[0, 1]**, чтобы корректно отобразить изображение:

```
filters_normalized = (filters - filters.min()) / (filters.max() - filters.min())
```

После этого выведите несколько фильтров в виде изображений (например, с помощью `matplotlib`).

3. **Прокомментируйте:** какие паттерны вы видите? Напоминают ли фильтры детекторы границ, градиентов, цветовых переходов?

Часть В. Визуализация карт активаций

1. **Выберите 2–3 изображения** из тестовой выборки (желательно из разных классов, одно из которых модель классифицирует правильно, другое — неправильно).
2. **Пропустите каждое изображение через модель**, извлекая промежуточные активации после одного-двух сверточных слоев (например, после первого и после более глубокого слоя). Для извлечения можно использовать forward hooks:

```
activations = {}

def hook_fn(name):
    def hook(module, input, output):
        activations[name] = output.detach().cpu()
    return hook
```

Пример регистрации хуков для **ResNet**:

```
model.layer1.register_forward_hook(hook_fn('layer1'))
model.layer3.register_forward_hook(hook_fn('layer3'))
```

Для других архитектур названия слоёв могут отличаться.

Чтобы посмотреть структуру модели, используйте:

```
print(model)
```

После регистрации хуков выполните прямой проход:

```
with torch.no_grad():
    output = model(image_tensor.unsqueeze(0))
```

3. **Визуализируйте карты активаций.** Извлеките несколько каналов (8–16) и выведите их в виде сетки изображений. Каждый канал соответствует одной **карте активации** (grayscale-изображению).
4. **Прокомментируйте полученные результаты:** что «видят» ранние слои (низкоуровневые признаки: границы, текстуры) и что — глубокие (высокоуровневые: части объектов, формы)?

Часть С. Grad-CAM

Grad-CAM (Gradient-weighted Class Activation Mapping) — метод визуализации, показывающий, какие области изображения наиболее важны для решения модели о конкретном классе.

Grad-CAM применим только к **сверточным архитектурам**.

1. **Реализуйте Grad-CAM** для последнего сверточного слоя модели. Алгоритм:
 - Выполните прямой проход, сохранив активации последнего сверточного слоя (через hook).
 - Выполните обратный проход (`.backward()`) от логита предсказанного класса, сохранив градиенты активаций (через hook на градиенты).
 - Усредните градиенты по пространственным измерениям (Global Average Pooling градиентов) — получите вектор весов для каждого канала.
 - Взвешенная сумма карт активаций с этими весами → тепловая карта.
 - Примените ReLU (оставьте только положительные значения).
 - Масштабируйте тепловую карту до размера исходного изображения (`F.interpolate` или `cv2.resize`).

Пример структуры:

```
import torch.nn.functional as F

gradients = {}
activations = {}

def save_activation(name):
    def hook(module, input, output):
        activations[name] = output.detach()
    return hook

def save_gradient(name):
    def hook(module, grad_input, grad_output):
        gradients[name] = grad_output[0].detach()
    return hook

target_layer = ... # последний сверточный слой модели

target_layer.register_forward_hook(save_activation("target"))
target_layer.register_full_backward_hook(save_gradient("target"))

model.eval()

output = model(image_tensor.unsqueeze(0))
```

```

pred_class = output.argmax(dim=1).item()

model.zero_grad()
output[0, pred_class].backward()

grads = gradients["target"]
acts = activations["target"]

weights = grads.mean(dim=(2, 3), keepdim=True)
cam = (weights * acts).sum(dim=1, keepdim=True)

cam = F.relu(cam)
cam = F.interpolate(cam, size=(image_height, image_width),
                    mode="bilinear", align_corners=False)

cam = cam.squeeze().cpu().numpy()
cam = (cam - cam.min()) / (cam.max() - cam.min() + 1e-8)

```

2. Наложите тепловую карту на исходное изображение:

```

plt.imshow(original_image)
plt.imshow(cam, cmap='jet', alpha=0.5)
plt.title(f"Предсказание:{pred_class_name}, Истинный:{true_class_name}")
plt.colorbar()
plt.show()

```

3. Визуализируйте Grad-CAM для не менее чем 8 изображений:

- 4 изображения, классифицированных **правильно** (из разных классов).
 - 4 изображения, классифицированных **неправильно**.
4. Дополнительно (по желанию): для одного изображения постройте Grad-CAM **для нескольких классов** (предсказанного и истинного, если они различаются) — покажите, на что модель «смотрит» при оценке разных классов.

Часть D. Анализ ошибок

1. Постройте матрицу ошибок (confusion matrix) на тестовой выборке:

```

from sklearn.metrics import confusion_matrix, ConfusionMatrixDisplay
import matplotlib.pyplot as plt

all_preds = []
all_labels = []

model.eval()

with torch.no_grad():
    for x_batch, y_batch in test_loader:

        x_batch = x_batch.to(device)

        logits = model(x_batch)
        preds = logits.argmax(dim=1)

        all_preds.extend(preds.cpu().numpy())

```

```
all_labels.extend(y_batch.cpu().numpy())

cm = confusion_matrix(all_labels, all_preds)

disp = ConfusionMatrixDisplay(cm, display_labels=class_names)

disp.plot(figsize=(10, 10), xticks_rotation=45)
plt.title("Confusion Matrix")
plt.show()
```

2. **Определите наиболее частые ошибки:** какие пары классов модель путает чаще всего? Выпишите топ-3 пары.
3. **Для каждой из трёх самых частых пар ошибок** покажите 2–3 примера неправильно классифицированных изображений с Grad-CAM. Прокомментируйте:
 - Почему модель могла ошибиться?
 - На какую область изображения модель «смотрела»?
 - Объективно ли сложен пример (нетипичный ракурс, плохое качество, похожие классы)?